

Freight Rate Prediction: Approach and Validation Report

Ahmad Arafat · Machine Learning Engineer assessment, Spotter · October 2026

1 Summary

The final model predicts load rates with about 1.6% MAPE on a two-months-ahead holdout, against 3.6% for a simple baseline. It combines a linear model for time-driven effects with LightGBM for load characteristics.

The training data covers January to October 2025 and the predictions are for November and December. So this is a forecasting problem, and most of my decisions follow from that: how I split the data, which features I kept, and how the model is built.

Table 1: Model comparison. Train on January–August, test on September–October.

Model	MAPE	MAE (\$)	RMSE (\$)
Baseline: median rate per mile by equipment and distance band	3.60%	79.0	107.5
Plain LightGBM, all features as tree inputs	1.88%	43.5	63.0
Hybrid: linear time component + LightGBM	1.67%	37.6	54.1
City-aware hybrid (final)	1.60%	35.9	51.8

2 Data exploration and quality issues

I found eight data-quality issues. Each one was investigated before deciding what to do, and every fix is applied the same way to the training data, the validation file and the December inputs.

Table 2: Data-quality issues, evidence and decisions.

Issue	Evidence	Decision
Corrupted labels (677 rows, 1.4%)	Rate per mile forms separate clusters at about 0.3x and 3.5x the normal level, with empty gaps between them. The share is flat across months and equipment, so it is not market behaviour.	Dropped from training (rate per mile below 1.4 or above 3.8)
Negative weights (292 train, 145 validation)	They mirror the positive distribution, including 13 rows at exactly $-47,500$, the same value as the upper cap	Sign errors: absolute value
Weight capped at 47,500, floored at 5,000	Spikes at exactly those values	Kept as-is; true value can't be recovered
Missing weight (0.6% train, 1.4% val.)	Missing at random, no difference in price	Training median
Missing market_index (0.8% train, 2.1% val.)	Missing at random; about 98% of its variance is day-level	Mean of the same date
Distance floored at 70 miles	48 rows at exactly 70; their prices fit other short hauls	Kept as-is
Perturbed, clipped coordinates	Not real geography (New Orleans and Shreveport are 7 miles apart); longitude clipped at -69.5 , latitude at 25.5	Used only as approximate location
quote_signal	Relationship with price flips between strongly positive and strongly negative across months (Section 3.3)	Dropped

One decision changed during the analysis. I first expected a lane-relative rule (rate compared to the lane median) to catch outliers better than a global threshold. When I compared them, they disagreed on 7 rows, all on lanes with only 1 to 9 loads. On such small lanes the median is pulled by the outlier itself, so the lane rule flagged clean loads and missed bad ones. The global threshold was right in all 7 cases, so I kept it.

3 Findings that shaped the model

Three findings decided the model design: price drifts upward over time on top of the market index, it ramps up at every quarter end, and `quote_signal` is unreliable.

3.1 Price depends on time, not only on the market index

To compare days fairly, I first removed the effect of distance and equipment (each load's rate per mile divided by the median for its equipment and distance band). The daily price level then rises from about 0.94 in January to 1.07 in late June, drops in July and August, and climbs again in September and October.

`market_index` explains part of this, but not all of it. At the same index value, later months are clearly more expensive: at an index of 0.9, January sits around 0.945 while September and October sit around 1.00 (Figure 1).

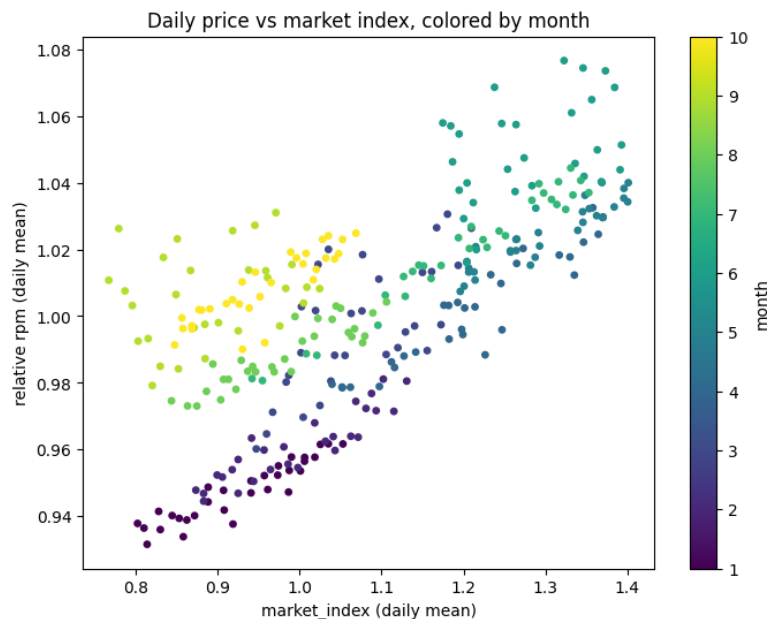


Figure 1: Daily mix-adjusted price level against the daily market index, coloured by month.

A linear fit on the daily data puts numbers on it. `market_index` alone explains 49% of the daily variation, time alone 27%, and both together 84.5%. The time trend is about +0.02% per day, or roughly +6.5% over the ten months.

This matters a lot for the forecast. The November and December `market_index` (0.72 to 1.10) looks like January's. A tree model can't extrapolate a time trend, so it would map November and December to January price levels and under-predict.

3.2 Prices ramp up at every quarter end

After removing `market_index` and the trend, the remaining daily pattern is a clear sawtooth (Figure 2). In March, June and September the price climbs steadily to about +3.7% by the last day of

the month, then drops back on the first day of the next quarter. The other months are flat.

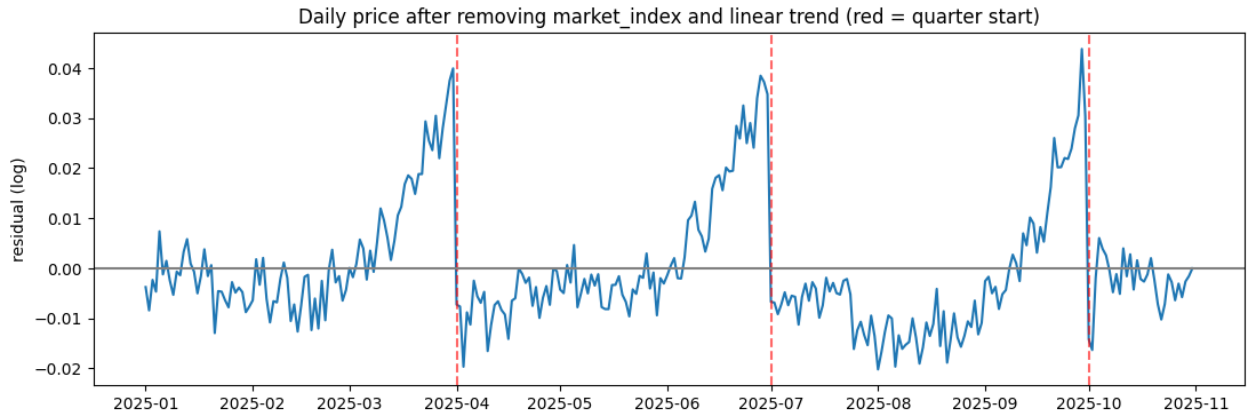


Figure 2: Daily price after removing market_index and the linear trend. Red lines mark quarter starts.

The ramp size is almost identical in all three quarters (+3.7%, +3.6%, +3.7%). December is a quarter-end month, so the model needs to reproduce this ramp.

3.3 quote_signal looks weak but is actually a trap

Overall, quote_signal has a correlation of only 0.10 with rate per mile. That number hides three different regimes (Figure 3):

- January, February, March, June and September: quote_signal is almost exactly the rate per mile (correlation +0.997).
- April, May, July and October: the relationship is inverted (correlation -0.994).
- August: no relationship at all (-0.02).

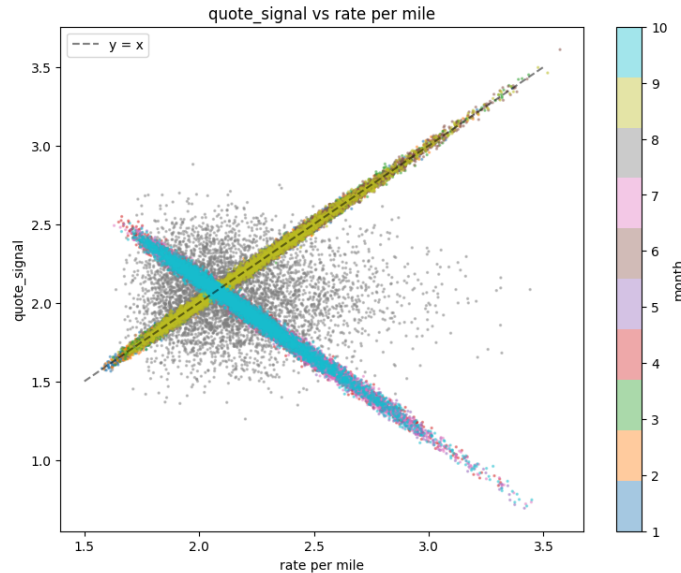


Figure 3: quote_signal against rate per mile, coloured by month.

In November and December its correlation with expected price is about zero (-0.001 and -0.018), so it behaves like August. A model would learn to rely on it, because it is nearly perfect for half the year, and then produce noisy predictions exactly where it matters. It also fools a naive holdout: September is one of the leaky months, so a September–October holdout would wrongly reward adding quote_signal. I dropped it.

3.4 12% of validation loads involve cities never seen in training

The validation file has 8 new cities (Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego), touching 1,447 loads. For those loads the model can't use city identity and has to rely on distance and coordinates.

4 Validation and split approach

I validated on time-based splits designed to mirror the real prediction setting: two months ahead, a quarter-end month in the test period, and cities the model has never seen.

4.1 Why not a random split

A random split puts loads from the same days and weeks in both train and test. The model then sees the market level of each test day through its neighbours, which never happens in production. It would also hide the two risks that matter most here: extrapolating the time trend and handling new cities. A random split would report an error that is too optimistic and would favour models that don't forecast well.

4.2 Holdout 1: time

I train on January to August and test on September to October. This copies the real gap (training ends in October, predictions cover November and December). September is a quarter-end month, so the test also checks whether the ramp is learned from March and June alone. Results are in Table 1.

4.3 Holdout 2: time plus unseen cities

Same split, but I also remove 8 randomly chosen cities from training entirely, matching the 8 new cities in the validation file. 26% of test loads touch these cities.

Table 3: Time plus unseen-city holdout (MAPE). 8 cities removed from training.

Model	All	Seen cities	Unseen cities
Plain LightGBM	1.97%	1.92%	2.11%
Hybrid	1.74%	1.69%	1.88%
Hybrid + city IDs	1.78%	1.62%	2.23%
City-aware hybrid	1.69%	1.62%	1.88%

This test changed the final model. City IDs help on known cities (1.69% to 1.62%) but hurt badly on new ones (1.88% to 2.23%), because the model learns to lean on the city name instead of the coordinates. The city-aware model routes each load to the right sub-model and gets the best of both.

4.4 Rolling backtest

One holdout period could be lucky, so I repeated the two-months-ahead test from three starting points, each time training on everything before the start.

The final model wins in every period, so the ranking is stable. July to August is harder because the market dropped abruptly in late July, which no model could anticipate. Based on this, I expect roughly 1.6% to 1.9% MAPE on November and December.

Table 4: Rolling backtest (MAPE). Each period is tested after training on all earlier data.

Test period	Plain LightGBM	Hybrid	City-aware hybrid
May to June	2.20%	1.64%	1.61%
July to August	2.12%	1.93%	1.90%
September to October	1.88%	1.67%	1.60%

4.5 Metrics and baselines

I report MAPE as the main metric because rates range from about \$185 to \$7,500, and a \$50 error means very different things on short and long hauls. MAE and RMSE are reported alongside it in dollars. Every model is compared against a simple baseline (median rate per mile by equipment and distance band) and a plain LightGBM, so each improvement can be traced to a specific decision. Corrupted labels are removed from the test sets too, so the scores measure real pricing error.

5 Model

The final model splits $\log(\text{rate})$ into a linear part for time-driven effects and a LightGBM part for everything about the load itself:

$$\log(\text{rate}) = \underbrace{\beta_1 \text{market_index} + \beta_2 t + \beta_3 \text{qe_day}}_{\text{linear}} + \underbrace{f(\text{distance, weight, equipment, coordinates})}_{\text{LightGBM}}$$

- **Log target.** Equipment effects are multiplicative and constant across distances (Reefer is 1.127x to 1.131x Dry Van in every distance band). In log space they become additive and easier to learn.
- **Linear part.** t is days since 1 January 2025 and qe_day is the day of month in March, June, September and December (0 otherwise). A linear model extends the trend and the quarter-end ramp into November and December, which trees can't do. That's the main reason the hybrid beats plain LightGBM.
- **LightGBM part.** Distance, log distance, weight, equipment and the four coordinates. These relationships are non-linear (rate per mile falls with distance because of fixed per-load costs) and their range doesn't change between train and validation, so trees are a good fit.
- **Backfitting.** The two parts influence each other: a day with many long hauls could look like a cheap market day. So I fit them alternately for three rounds, each part learning from what the other left unexplained, the same idea as in generalized additive models.
- **City-aware routing.** Two hybrids are trained, one with pickup and delivery city as categorical features and one without. A load uses the first only when both its cities were seen in training. In the validation file, 87.9% of loads use the city model and 12.1% use the fallback.

The fitted coefficients match what the exploration showed: +0.142 per unit of `market_index`, +0.020% per day of trend, and +0.127% per day of the quarter-end ramp (about +3.9% by the 31st).

6 December prediction

For the fixed Lexington to Fort Wayne load, the predicted rate rises from \$830.60 on 1 December to \$891.87 on 31 December (+7.4%), with a weekly cycle on top.

The December input file has no coordinates or `market_index`, so I filled them in. Coordinates come from a city lookup built from the training and validation files (both cities appear in training).

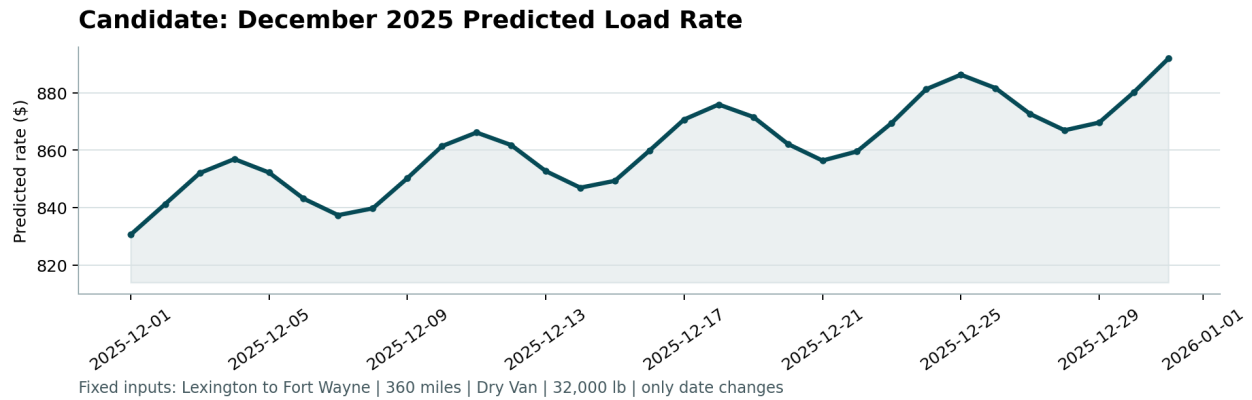


Figure 4: December 2025 predicted rate, generated by the provided `score.py`.

`market_index` is the mean of all validation loads on the same date, which is consistent with how missing values are handled elsewhere and uses only features, not labels.

The chart shows three things the model learned:

- **Weekly cycle.** Every peak falls on a Thursday (4, 11, 18, 25 December) and every low on a Sunday. This comes from the weekly pattern in `market_index`.
- **Quarter-end ramp.** Each peak and each low is higher than the one before. The ramp adds about 3.9% by the 31st.
- **End of quarter beats the weekly cycle.** 31 December is a Wednesday, yet it's the highest point of the month, the same pattern seen at the end of March, June and September.

7 Assumptions and limitations

The forecast rests on three assumptions I can't verify from the training data, plus one risk outside the model's control.

- **The trend continues.** The linear trend is extended two months past the last training date. This is the weakest assumption. A sudden market shift like the one in late July would push errors toward the upper end of the backtest range.
- **The quarter-end ramp repeats in December.** It appeared in all three observed quarter ends with almost the same size, so I expect it again, but December is the first year-end quarter in the data.
- **No holiday effect.** I found no price change around 4 July or Labor Day, so the model has no holiday adjustment. Real freight markets usually soften around Christmas, so the 25 December prediction, a weekly peak, is probably too high.
- **Corrupted labels in validation.** If the validation labels contain the same 1.4% corruption as training, those loads will have very large errors that no model can avoid. Low outliers hurt most: predicting a normal rate for a load recorded at 0.3x gives an error above 200%. My 1.6% to 1.9% estimate is for clean labels only.

With more time, I would add holiday features using several years of history, test non-linear forms of the trend, and check how far the trend can be extrapolated before it degrades.